

AI Personal Wardrobe & Virtual Fashion Platform

Technology Stack Reference

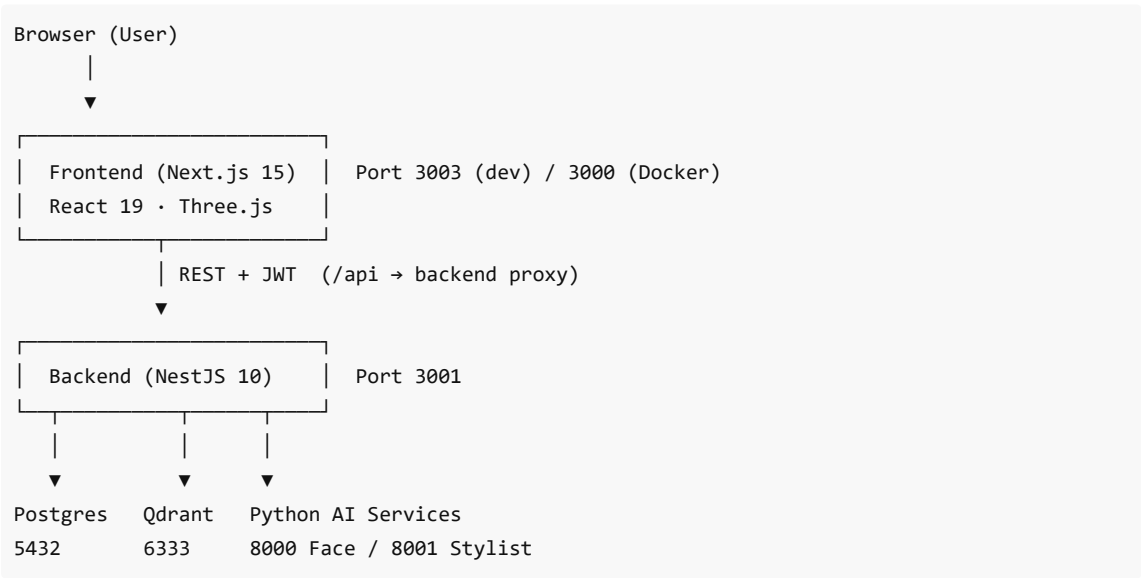
Document version: 2.0
Date: June 22, 2026
Project: wardrobe-ai/ monorepo

1. Executive Summary

This platform is a **multi-service, Docker-first** application for AI-powered wardrobe management, face authentication, outfit styling, **3D virtual try-on**, e-commerce catalog flows, and social sharing. It combines a modern JavaScript web stack with Python microservices for machine learning workloads, backed by **PostgreSQL** for relational data and **Qdrant** for vector search.

Layer	Technology	Role
Frontend	Next.js 15 + React 19	User interface, 3D viewer, camera auth
API Gateway	NestJS 10 + TypeScript	REST API, auth, orchestration
AI Services	FastAPI + Python	Face embeddings, clothing analysis
Databases	PostgreSQL 16 + Qdrant	Relational data + vector similarity
Infrastructure	Docker Compose	Local & production container stack

2. Architecture Overview



Six core Docker services: frontend , backend , postgres , qdrant , face-service , stylist-service — all on internal network wardrobe-network .

3. Languages & Runtimes

Language / Runtime	Where Used	Purpose
TypeScript 5.7	backend/src/	Type-safe API, DTOs, services
JavaScript (JSX)	frontend/src/	Next.js App Router UI
Python 3	ai-services/	ML inference (InsightFace, PyTorch, rembg)
SQL	database/postgres/	Schema, migrations, seed data
YAML / Shell / PowerShell	docker-compose.yml, scripts/	Infrastructure & automation

4. Frontend Stack (frontend/)

4.1 Core Framework

Technology	Version	Used For
Next.js	15.x	App Router, SSR, API rewrites to backend, image optimization, code splitting
React	19.x	Component-based UI across all features
Tailwind CSS	3.4.x	Utility styling — Deep Obsidian + Vivid Magenta design system
PostCSS + Autoprefixer	—	CSS processing pipeline

4.2 UI Components & Design System

Library	Used For
Radix UI (dialog, tabs, select, alert-dialog, progress, slot)	Accessible, unstyled primitives for modals, forms, navigation
class-variance-authority (CVA)	Variant-based button/badge styling
clsx + tailwind-merge	Conditional + deduplicated Tailwind class names (ShadCN pattern)
Lucide React	Icons across nav, catalog, auth, admin, share menu
next-themes	Dark/light mode with SSR-safe theme switching
tailwindcss-animate	Enter/exit animations for modals and menus

4.3 State Management & Data Fetching

Library	Used For
Zustand	Client state — auth session, cart, wishlist, face studio, toasts, orders

TanStack React Query	Server state — wardrobe, outfits, profile, catalog products, dashboard
React Hook Form	Login, registration, profile, settings forms
Zod + @hookform/resolvers	Schema validation for forms and onboarding

4.4 3D Virtual Try-On

Library	Used For
Three.js	WebGL 3D rendering engine
@react-three/fiber	React renderer for Three.js scenes
@react-three/drei	useGLTF, Clone, camera controls, lighting helpers
GLB models (public/models/)	Shirt, jacket, blazer garment meshes on 3D mannequin
Custom ThreeDViewer	Garment fitting, face overlay from Face Studio, 360° spin

4.5 Face Authentication (Client)

Library	Used For
@vladmandic/face-api	Browser face detection during camera capture
TensorFlow.js (core + WebGL backend)	Hardware-accelerated client-side face inference
browser-image-compression	Compress captures before upload (~WebP, max dimension)
MediaDevices API	Camera stream for registration and face login

4.6 Charts & Admin

Library	Used For
Recharts	Admin console revenue donut, trend charts, product metrics

4.7 Commerce & Social

Feature	Technology	Used For
Cart / Wishlist	Zustand stores	Add-to-cart, save items, drawer UI
Checkout	React + Zustand	Order summary, payment method selection
Social Sharing	Web Share API + fallback modal	Share 3D try-on to Instagram, WhatsApp, Facebook, Pinterest

4.8 Frontend Feature Modules

Module	Path	Purpose
--------	------	---------

auth	features/auth/	Face + email login, registration, JWT session
landing	features/landing/	Marketing homepage
dashboard	features/dashboard/	Personalized home, recommendations, Fashion DNA
wardrobe	features/wardrobe/	Digital closet upload, grid, filters
outfits	features/outfits/	AI-generated outfit cards, feedback
catalog	features/catalog/	Product browse, filters, product cards
try-on	features/try-on/	Virtual Try-On modal, 3D viewer, share menu
face-studio	features/face-studio/	User face capture for 3D avatar overlay
closet	features/closet/	Style profile, polaroid wardrobe view
commerce	features/commerce/	Cart drawer, wishlist drawer
checkout	features/checkout/	Checkout flow
admin	features/admin/	Admin console (RBAC-gated)
profile	features/profile/	Onboarding, preferences
settings	features/settings/	Account settings
recommendations	features/recommendations/	Curated style carousels

5. Backend Stack (backend/)

5.1 Core Framework

Technology	Version	Used For
NestJS	10.x	Modular API — controllers, services, guards, DI
Express (via @nestjs/platform-express)	—	HTTP server, multipart file uploads
TypeScript	5.7.x	End-to-end type safety
RxJS	7.x	Async streams in NestJS internals

5.2 Authentication & Security

Library	Used For
@nestjs/jwt	Issue and verify JWT access tokens
Passport + passport-jwt	JWT strategy and route guards
bcryptjs	Hash passwords for email/password auth
class-validator + class-transformer	DTO validation via NestJS pipes

5.3 API & Documentation

Library	Used For
@nestjs/swagger	OpenAPI docs at /api/docs
@nestjs/config	Environment-based configuration

5.4 Data Access

Library	Used For
pg	PostgreSQL queries (users, wardrobe, outfits, products)
@qdrant/js-client-rest	Face vector search, fashion DNA / clothing embeddings

5.5 Backend Modules

Module	Used For
AuthModule	Face register/login, email auth, JWT
ProfileModule	User profile, onboarding, preferences
FashionDnaModule	Style DNA scoring and vectors
WardrobeModule	Clothing item CRUD, image uploads
OutfitsModule	Outfit generation, stylist orchestration
ProductsModule	E-commerce catalog API
TryOnModule	Virtual try-on session requests
ChatModule	AI stylist chat endpoints
DatabaseModule	Postgres connection pool

5.6 Testing

Library	Used For
Jest + ts-jest	Unit tests for services (outfits, wardrobe, products)
@nestjs/testing	NestJS test module harness

6. AI Services (ai-services/)

6.1 Face Service — Port 8000

Purpose: 512-dimensional ArcFace embeddings for face registration and login.

Library	Used For
---------	----------

FastAPI	REST API (/v1/face/embed, /health)
Uvicorn	ASGI production server
Pydantic	Request/response validation
InsightFace	State-of-the-art face recognition (buffalo_L model)
ONNX Runtime	Efficient CPU inference for face models
OpenCV (headless)	Image preprocessing, alignment
NumPy + Pillow	Array and image handling

Why separate service: Heavy ML models (~100MB+), best Python ecosystem support, isolated from Node restarts.

6.2 Stylist Service — Port 8001

Purpose: Clothing image analysis, outfit recommendations, background removal.

Library	Used For
FastAPI + Uvicorn + Pydantic	HTTP API layer
PyTorch + torchvision	MobileNetV2 category detection, 512-dim clothing embeddings
scikit-learn	K-Means dominant color extraction
rembg (U-2-Net)	Automatic background removal on wardrobe uploads
OpenCV + Pillow + NumPy	Image preprocessing pipelines
pytest + httpx	API integration tests

Key endpoints: /v1/clothing/analyze , /v1/outfits/recommend , /health

7. Databases & Storage

Technology	Version	Port	Used For
PostgreSQL	16 (Alpine)	5432	Users, profiles, wardrobe items, outfits, products catalog
Qdrant	1.12.x	6333	Face vectors, fashion DNA, recommendation & clothing embeddings
Docker volume	—	—	wardrobe-uploads-data — wardrobe image files

PostgreSQL Migrations

Migration	Purpose
001	Users table

002	User profile tables
003	Clothing items
004	Outfits
005	Products catalog
006	Catalog refresh seed
007	AI render image column

Qdrant Collections

Collection	Purpose
users_face_vectors	Face login similarity search
fashion_dna_vectors	Style DNA embeddings
recommendation_vectors	Recommendation engine
clothing_item_vectors	Wardrobe item similarity

8. DevOps & Tooling

Technology	Used For
Docker + Docker Compose	6-service reproducible stack
Multi-stage Dockerfiles	Optimized frontend & backend images
<code>.env / .env.local</code>	Secrets, ports, admin emails (never committed)
ESLint + Prettier	Linting and code formatting
knip	Unused dependency detection (frontend)

NPM Scripts (root)

Script	Purpose
<code>npm run dev</code>	Start frontend dev server (port 3003)
<code>npm run dev:backend</code>	Start NestJS in watch mode
<code>npm run infra:up</code>	<code>docker compose up -d</code>
<code>npm run db:migrate</code>	Run PostgreSQL migrations

9. Service URLs

Service	Dev URL	Docker URL
---------	---------	------------

Frontend	http://localhost:3003	http://localhost:3000
Backend API	http://localhost:3001/api	http://localhost:3001/api
Swagger UI	http://localhost:3001/api/docs	same
Face AI	http://localhost:8000	http://face-service:8000
Stylist AI	http://localhost:8001	http://stylist-service:8001
PostgreSQL	localhost:5432	postgres:5432
Qdrant	http://localhost:6333	qdrant:6333

10. Feature → Technology Map

Feature	Primary Technologies
Face registration & login	face-api.js, InsightFace, Qdrant, JWT, NestJS Auth
Wardrobe upload	Next.js, NestJS, PostgreSQL, stylist-service, rembg
Outfit recommendations	stylist-service PyTorch, NestJS OutfitsModule, React Query
3D Virtual Try-On	Three.js, R3F, drei, GLB models, Face Studio overlay
Product catalog	NestJS ProductsModule, PostgreSQL, Next.js catalog feature
Cart & checkout	Zustand, React, local order state
Admin console	Recharts, RBAC (NEXT_PUBLIC_ADMIN_EMAILS), guarded routes
Social sharing	Web Share API, ShareMenu modal, clipboard API
Fashion DNA / Closet	Qdrant vectors, dashboard widgets, radial score UI
AI stylist chat	NestJS ChatModule, frontend chat UI
Landing page	Next.js static sections, Playfair + mono typography
Image optimization	Next.js <code><Image></code> , browser-image-compression, WebP

11. Design Decisions (Why This Stack)

Decision	Why
Next.js over plain React SPA	Built-in routing, image optimization, API proxy rewrites
NestJS over raw Express	Modular architecture, guards, Swagger, scales with features
Python microservices for AI	Best ML libraries (InsightFace, PyTorch, rembg)
PostgreSQL + Qdrant	Relational integrity + fast local vector search without cloud
Zustand + React Query	Minimal boilerplate; clear client vs server state split

Three.js via R3F	React-friendly 3D for virtual try-on in browser
Docker Compose	Appropriate for local dev and MVP deployment
Radix + Tailwind	Accessible primitives with full design control